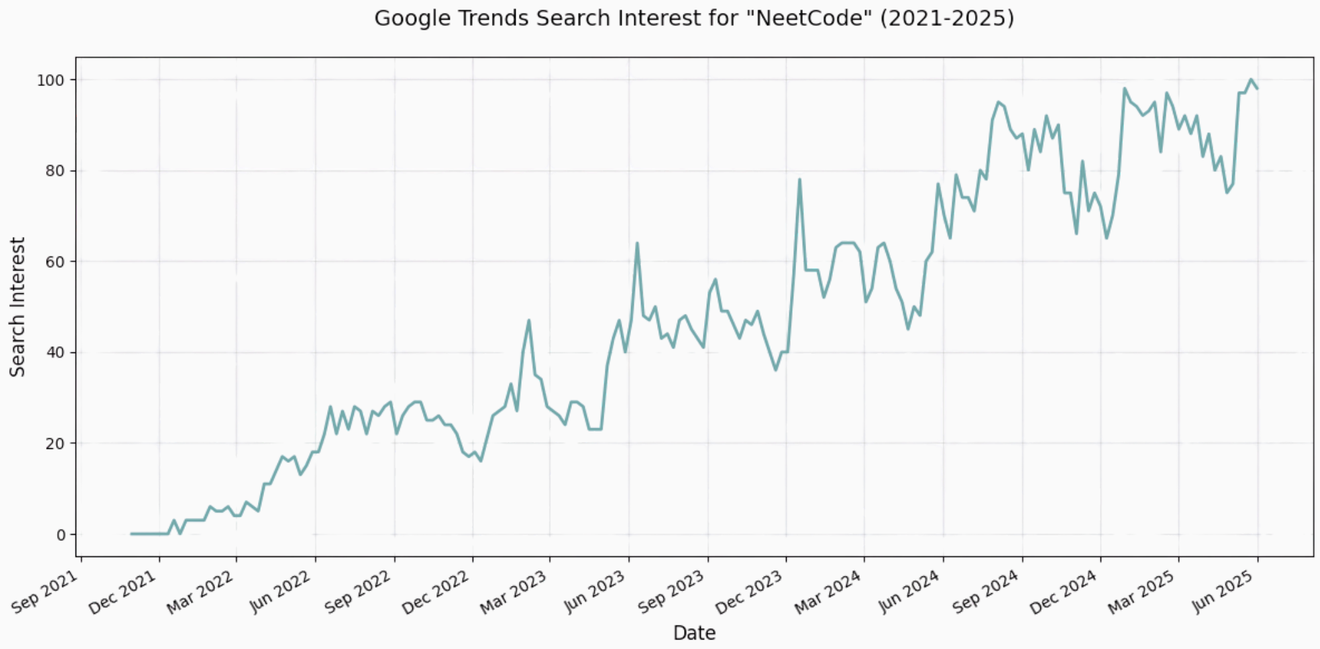
# An Actor-Critic Approach for the Recommendation of Algorithmic Programming Questions

Presentation By Dan Laurentiu

# Introduction

With the rise in popularity of algorithmic programming problems proposed in technical interviews for software engineering roles, developing a practical but effective learning plan is crucial. To the best of our knowledge, there is currently no automated tool to help with this.

This thesis aims to address this gap by proposing an Advantage Actor-Critic model (A2C), with the purpose of generating learning plans specifically for LeetCode problems. This model will be integrated into the backend of a web application, serving as a proof-of-concept. The application, a2cl, will allow for creating, monitoring, and tracking the learning process.



# Contributions

## 01 Application

Firstly, we introduce a2cl, a web-based tool designed to generate and track the progress of new learning plans.

## 02 Model

Secondly, we adapt an Advantage Actor-Critic approach for this specific application. To the best of our knowledge, this has never been experimented with before.

## 03 Data Analysis

Lastly, this work also includes a data analysis of LeetCode problems. This offers information that may inform future research.

# Literature

One of the most popular challenges with RL systems for recommendation is the impractical nature of online training.

RL approaches learn very slowly and are very sample inefficient, users will suffer through many bad recommendations before seeing a good one.

Because of this, all the approaches presented next work on improving offline training.

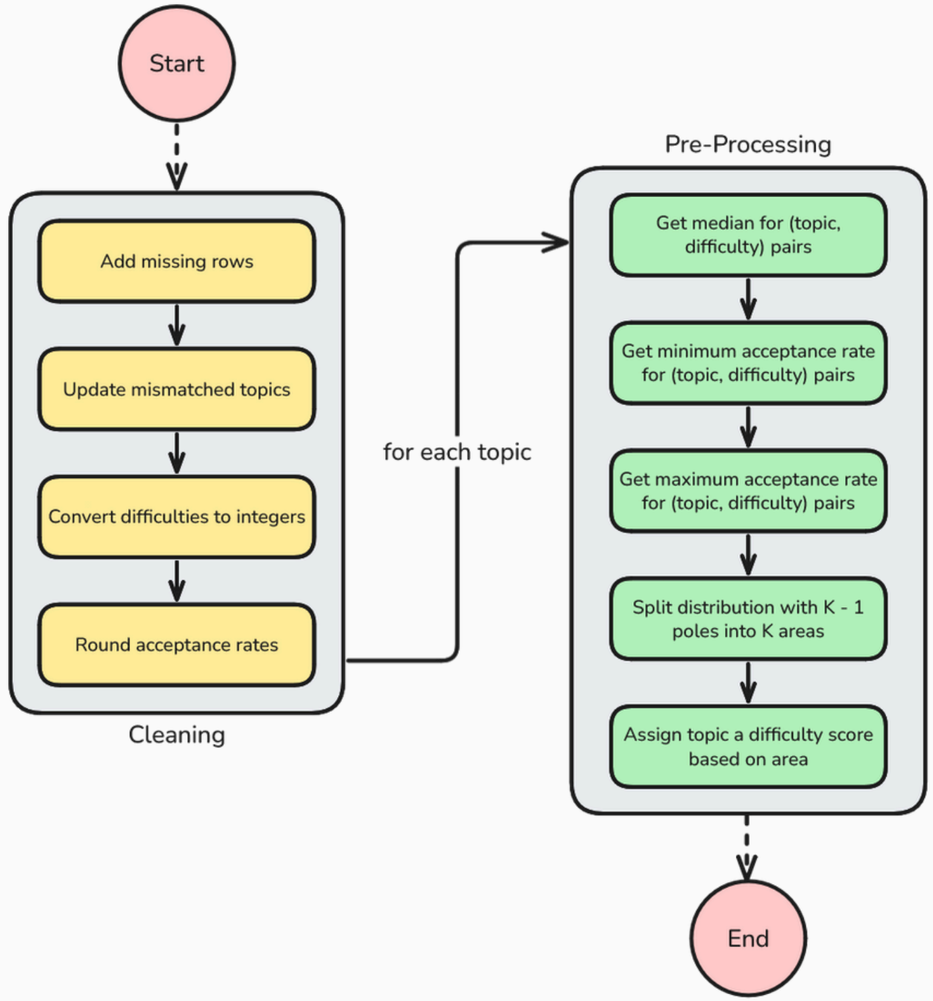
**01** In 2019, Xiangyu Zhao et al. used a Deep-Q-Learning (DQN) algorithm to recommend a list of items. The intuition was that by recommending a list of items, with potentially complementary items, it would model in some way the complex relationships between each item and be more flexible to a user. They compared it with popular recommendation models: CF, RNNs, FMs, along others. The long-term results achieved from 18%-32% increased reward-per-item.

**02** In 2020, Sijin Zhou et al. focused on improving the DQN algorithm. They worked on two areas: trying to get more context (better features) for the agent, and eliminating as much size from the state-space as possible. They used a Knowledge Graph (KG), parsed it with a Graph Convolutional Network (GCN) and then a Gated Recurrent Unit (GRU) created the agent's features. Additionally, the system gathers the k-hop KG neighborhood around a user’s positive clicks and forms a much smaller candidate pool. KGQR reaches 3–5% higher cumulative reward and moderate gains in precision and recall while using fewer than 20% of the interactions required by normal DQN.

# Methodology

My approach mirrors the first paper from the previous slide, also focusing on recommending lists of complementary items (actions). However, I used the A2C algorithm instead of DQN to provide more training stability.

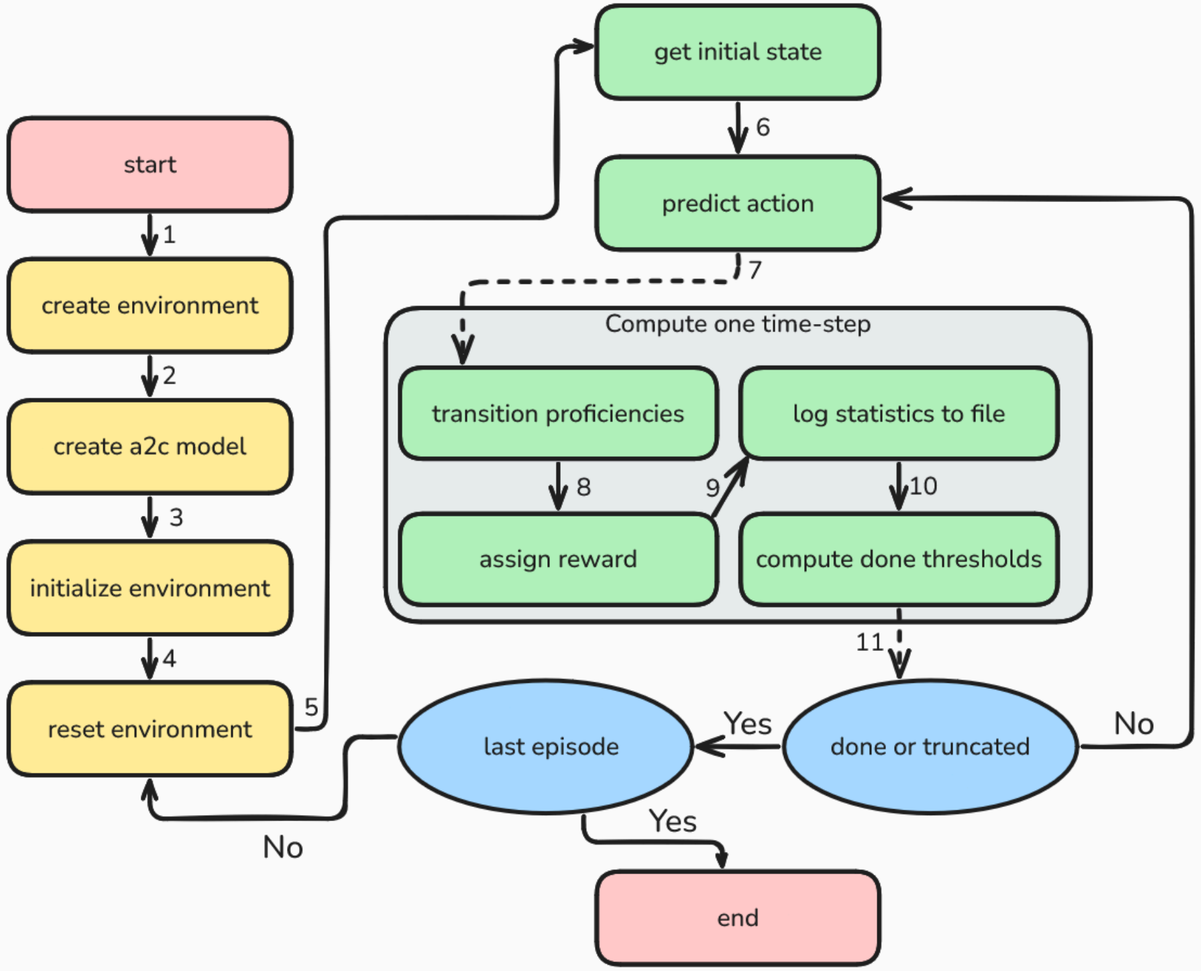
Also, I included a combination of statistical and heuristic methods to evaluate each topic from a problem, rather than the problem itself.



# Methodology

The critical decisions made in developing the environment were: state and action representation, reward scheme, state transition logic, episode termination logic and environment reset protocol.

| Scheme | Values |
|---|---|
| aggressive | (-20, +100, +20, +3, +2, +1, +1, +0, -1) |
| moderate | (-10, +100, +10, +3, +2, +1, +1, +0, -1) |
| mild | (-5, +100, +5, +3, +2, +1, +1, +0, -1) |
| repeated action over right topic | (-20, +100, +10, +3, +2, +1, +1, +0, -1) |
| right topic over repeated action | (-10, +100, +20, +3, +2, +1, +1, +0, -1) |
| RTORA + dense rewards | (-2, +100, +10, +2, +3, +1, +1, +0, -1, +sum) |

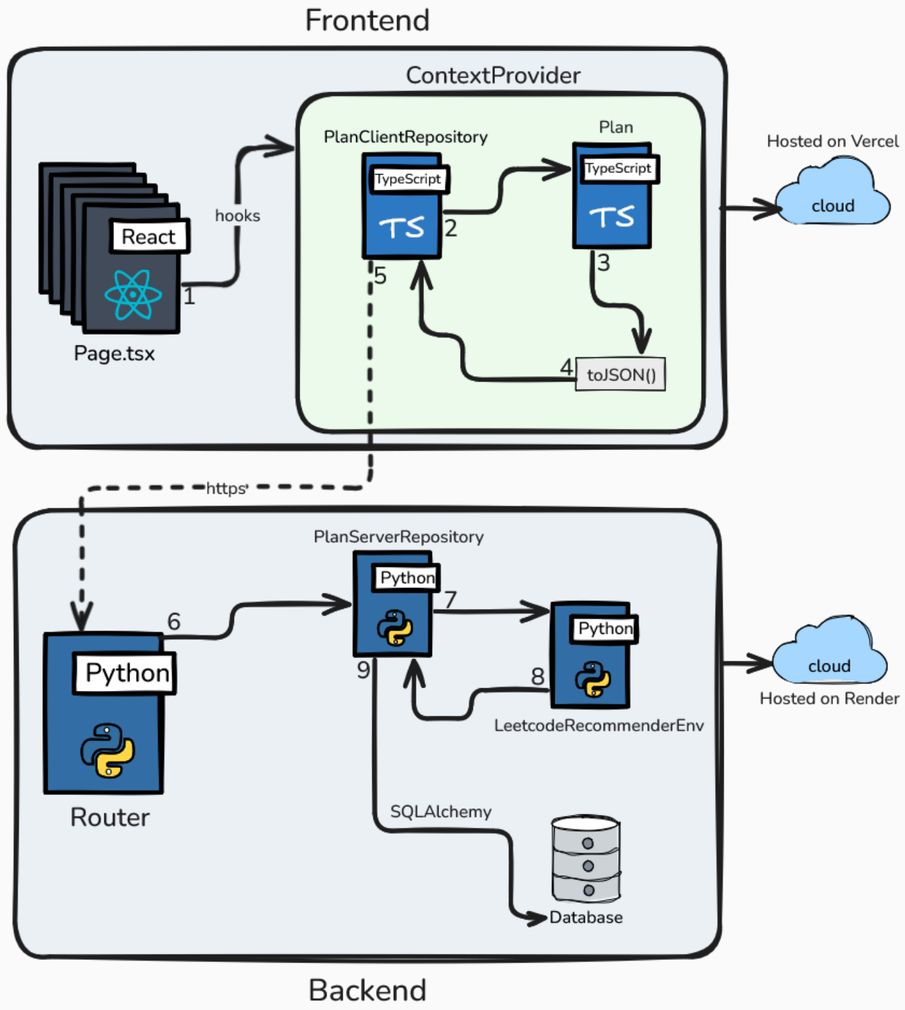


# Application

The application is built on a three-layer architecture. The frontend is developed using Next.js, a React-based framework, the backend is implemented with FastAPI and is responsible for hosting the reinforcement learning (RL) algorithm and environment, and interacting, via SqlAlchemy ORM, with the data layer. This data layer uses a PostgreSQL relational database to store information about: topics, problems, plans and their progress.

The front-end application is hosted on Vercel, a cloud platform optimized for Next.js and React applications. Vercel automatically builds and deploys every Git branch as a preview environment.

The backend server and database server are both hosted on Render, a modern cloud platform known for automatic deployments from GitHub branches and simplicity.



# Application

The backend exposes RESTful API endpoints which use Pydantic schemas for type-safe communication. This system also uses a seeding mechanism built alongside SQLAlchemy to populate the PostgreSQL database with all existing leetcode problems and topics. These seeders operate under two conditions: as a one-time initialization step during production deployment, or as an automatic repopulation mechanism during development whenever database tables are empty.

| PlanJSON Schema | |
|---|---|
| id | int |
| title | str |
| profile | ProfileJSON |
| problems | List[Tuple[bool, ProblemJSON]] |
| topics | Optional[List[TopicJSON]] |
| given_topics | List[TopicJSON] |

Frontend styling is done using TailwindCSS and the frontend is written in TypeScript. The graph structure used to display each problem in the Plan Screen is built using the ReactFlow library.

Some security measures are: encrypted communication using the HTTPS protocol and database query handling using prepared statements.

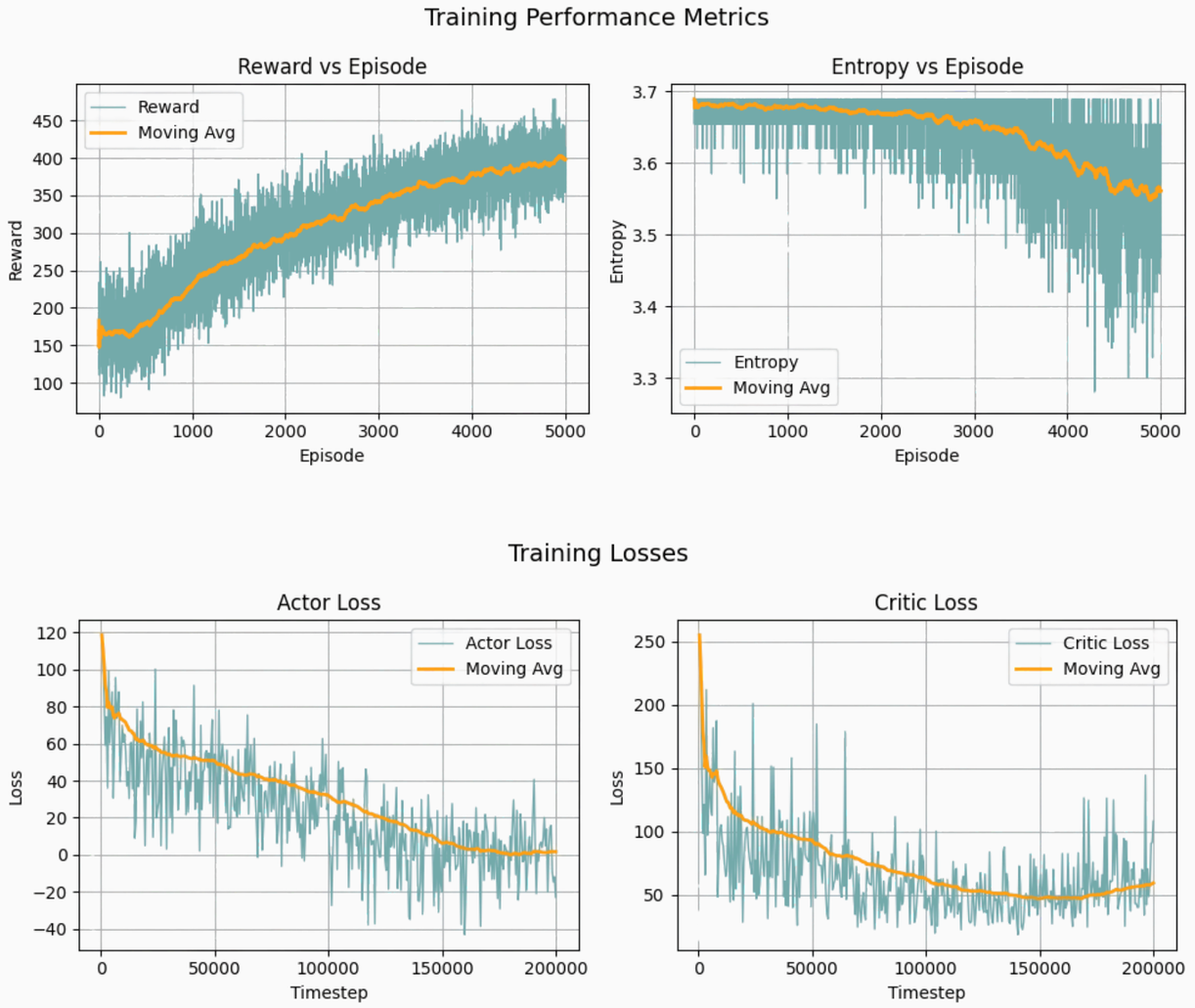
In both the frontend and backend, PlanValidator and ProfileValidator validate all fields of their corresponding classes.

We conducted a series of tests on our backend implementation. The unit tests achieve 100% decision coverage against: the domain, the repository and the JSON serialization logic. Additionally, exploratory testing validate the GET, POST, PATCH and DELETE endpoints.

# Results

The reward progression shows steady improvement. This entropy curve suggests an appropriate exploration-exploitation balance in the beginning, although the final entropy levels do not get to converge low enough.

The actor's gradient norms remain an order of magnitude smaller than the critic's, indicating a severe imbalance in learning signals. Notably, the actor's gradient vanishes after ten thousand time steps.



# Conclusion

The main contributions lie in framing problem recommendations as a sequential decision-making task. By using user-specific learning targets and statistical problem data, the proposed algorithm demonstrates relative feasibility. However, a constant challenge was the actor network's vanishing gradient. This heavily limited the model's ability to run for more episodes and improve the recommendations. Despite many attempts to solve this issue (gradient clipping, multiple weight initialization methods, simpler and more complex network architectures, different coefficient values for losses, different learning rates, different gradient descent optimizers, and using multiple feature extractors), the problem is still unresolved.

Firstly, developing a model-based RL environment with many parallelized simulations will more efficiently mimic real user interactions. Also, making sparse use of online algorithms to gather significant context via limited real-user feedback would drastically improve reliability.

Secondly, generating rich embeddings for each LeetCode question, by using knowledge graphs (KGs) and graph convolutional networks (GCNs) holds much promise. However, constructing LeetCode-specific KGs requires significant effort, which was well past the scope of our work.

# Thank You!